

Object-Oriented Programming Exercises

4 Developing the CloudSharing application

4.1 Problem Description

The goal of this project is the development of an application that allows the management of a system to store files in the cloud (e.g. Dropbox). Each registered user has access to an account where he can add files. This account has a maximum capacity that cannot be exceeded. Files stored on the system can be accessed through a website. In addition, it is also possible to share files with other registered users.

Each account (or user) has an email (unique account identifier) and the files it owns. There are two types of accounts, free basic accounts (“basic”) and premium accounts with annual fee (“premium”). Premium accounts have 5GB of space and can share their files with other registered users. Basic accounts have 2GB of space and your files cannot be shared. **Remember that 1GB corresponds to 1024MB.** Another difference between basic and premium accounts is the accounting for used space. In premium accounts, the space occupied by shared files is not counted, while in basic accounts the space occupied by shared files is counted at 50%. For example, for a basic account with 100MB of proprietary files and 80MB in shared files, this account is considered to use 140MB of the available 2GB.

Each file has a name, a size (in MB) and an owner account (or user). A file is uniquely identified by its name and the account to which it belongs.

The application must then allow:

1. Each account (or user) has an email (unique account identifier) and the files it owns. There are two types of accounts, the free basic accounts Create an account (command **ADD**). The name of the account (the user’s email) and the type of account (basic or premium) are provided. In case of success, the account is added (“Account was added.”). The operation fails if: (1) an account with that email already exists (“Account already exists.”).
2. Add a file (command **UPLOAD**). Account name, file name and file size in MB are provided. In case of success, the file is added to the account (“File uploaded into account.”). The operation fails if: (1) the account does not exist (“Account does not exist.”); (2) a file with that name already exists in that account (“File already exists in the account.”); or (3) the file size exceeds account capacity (“File size exceeds account capacity.”).
3. Share a file (**SHARE** command). The name of the account the file belongs to, the name of the account to add to the share, and the name of the file are provided. In case of success, the file is shared (“File was shared.”). The operation fails if: (1) one of the accounts does not exist (“Account does not exist.”); (2) the file does not exist (“File does not exist.”); (3) the account does not allow file sharing (“Account does not allow file sharing.”); (4) the share already exists (“File already shared.”); or (5) the file size exceeds the account capacity (“File size exceeds account capacity.”).

4. List all owned and shared files of an account (`LISTFILES` command). The account name is provided. If successful, the operation displays the header (“Account files:”) followed by the name of each file, its size and whether it is shared. The operation fails if: (1) the account does not exist (“Account does not exist.”).
5. List all accounts (`LISTALL` command). The operation is always successful and displays the header (“All accounts:”) followed by the name of each account and the type of account.
6. Exit the application (command `EXIT`). The operation is always successful (“Exiting...”).

4.2 Example of interaction with the CloudSharing application

Develop your application so that it guarantees, at least, the interaction model illustrated in the following example (the character `↵` represents a line break):

```
ADD cf@gmail.com premium↵
Account was added.↵
ADD cf@gmail.com basic↵
Account already exists.↵
UPLOAD cf@gmail.com poo.txt 2↵
File uploaded into account.↵
UPLOAD poo@gmail.com poo.txt 2↵
Account does not exist.↵
UPLOAD cf@gmail.com poo.txt 2↵
File already exists in the account.↵
UPLOAD cf@gmail.com ada.pdf 7000↵
File size exceeds account capacity.↵
UPLOAD cf@gmail.com contactBook.pdf 3000↵
File uploaded into account.↵
ADD mg@fct.unl.pt basic↵
Account was added.↵
ADD am@fct.unl.pt premium↵
Account was added.↵
SHARE cf@gmail.com am@fct.unl.pt poo.txt↵
File was shared.↵
UPLOAD am@fct.unl.pt stupidFriendsBook.pdf 2000↵
File uploaded into account.↵
SHARE am@fct.unl.pt bf@fct.unl.pt zoo.zip↵
Account does not exist.↵
UPLOAD mg@fct.unl.pt zoo.zip 1000↵
File uploaded into account.↵
SHARE mg@fct.unl.pt am@fct.unl.pt zoo.zip↵
Account does not allow file sharing.↵
SHARE cf@gmail.com mg@fct.unl.pt contactBook.pdf↵
File size exceeds account capacity.↵
SHARE cf@gmail.com am@fct.unl.pt poo.txt↵
File already exists in the account.↵
LISTFILES am@fct.unl.pt↵
Account files:↵
poo.txt (2 MB) (shared)↵
stupidFriendsBook.pdf (2000 MB)↵
```

```
LISTALL ↵  
All accounts:↵  
cf@gmail.com (Premium)↵  
mg@fct.unl.pt (Basic)↵  
am@fct.unl.pt (Premium)↵  
EXIT↵  
Exiting...↵
```

4.3 Development

Develop your application according to the following phases:

1. Develop the application support interface(s). Use the concept of polymorphism. Draw a diagram of classes and interfaces to illustrate the modeling proposal.
2. Deploy and test the application.
3. Submit the application source code to *Mooshak*.

Test files

Mooshak's tests incrementally check the implementation of the various commands:

- Test file: `1_in_base_add.txt` (20 points)
Tested commands: `ADD`, `LISTALL`, `EXIT`
Context: Conditions where commands may fail are not tested.
- Test file: `2_in_base_upload.txt` (20 points)
Tested commands: all commands from test 1 and the `UPLOAD`, `LISTFILES` commands
Context: Conditions where commands may fail are not tested.
- Test file: `3_in_base_share.txt` (20 points)
Commands tested: all commands from test 2 and the command `SHARE`
Context: Conditions where the command might fail are not tested.
- Test file: `4_in_pre_add.txt` (10 points)
Tested commands: `ADD`
Context: All conditions under which the `ADD` command may fail are tested.
- Test files: `5_in_pre_upload.txt` (10 points)
Tested commands: `UPLOAD`
Context: All conditions under which the `UPLOAD` command might fail are tested.
- Test files: `6_in_pre_share.txt` (10 points)
Tested commands: `SHARE`
Context: All conditions under which the `SHARE` command might fail are tested.
- Test file: `7_in_pre_files.txt` (10 points)
Tested commands: `LISTFILE`
Context: All conditions under which the `LISTFILES` command may fail are tested.